Practical 1: Count lines and characters in an input file

Input: Uses `prac1_input.txt` (two lines of text).

Build:

```
flex -o lex.yy.cpp cd_prac_1.l
g++ -std=c++17 lex.yy.cpp -o practical_1
```

Run:

```
./practical_1 < prac1_input.txt
```

Input file (`prac1_input.txt`):

```
This is the input file for practical 1.
It has two lines of text.
```

Lex program (`cd_prac_1.l`):

```
%{
#include <iostream>
using namespace std;

int character_count = 0;
int line_count = 0;
bool last_character_was_newline = true;
%}

%option noyywrap

%%
\n      { ++character_count; ++line_count; last_character_was_newline =
true; }
.       { ++character_count; last_character_was_newline = false; }
%%

int main()
{
    yylex();
    if (character_count > 0 && !last_character_was_newline)
        ++line_count;

    cout << "Lines: " << line_count << '\n';
```

```
    cout << "Characters: " << character_count << '\n';
    return 0;
}
```

Output:

```
Lines: 2
Characters: 66
```

Practical 2: Count vowels and consonants in a string

Input: Input: `Hello, world! AEIOU`

Build:

```
flex -o lex.yy.cpp cd_prac_2.l
g++ -std=c++17 lex.yy.cpp -o practical_2
```

Run:

```
printf 'Hello, world! AEIOU\n' | ./practical_2
```

Lex program (`cd_prac_2.l`):

```
%{
#include <iostream>
using namespace std;

int vowel_count = 0;
int consonant_count = 0;
%}

%option noyywrap

%%
[aeiouAEIOU]  { ++vowel_count; }
[a-zA-Z]      { ++consonant_count; }
.             { }
\n            { }
%%

int main()
{
    yylex();
```

```
    cout << "Vowels: " << vowel_count << '\n';
    cout << "Consonants: " << consonant_count << '\n';
    return 0;
}
```

Output:

```
Vowels: 8
Consonants: 7
```

Practical 3: Caesar cipher with a shift of three

Input: Input: `xyz XYZ`

Build:

```
flex -o lex.yy.cpp cd_prac_3.l
g++ -std=c++17 lex.yy.cpp -o practical_3
```

Run:

```
printf 'xyz XYZ\n' | ./practical_3
```

Lex program (`cd_prac_3.l`):

```
%{
#include <iostream>
using namespace std;
%}

%option noyywrap

%%
[a-z]         { cout.put('a' + (yytext[0] - 'a' + 3) % 26); }
[A-Z]         { cout.put('A' + (yytext[0] - 'A' + 3) % 26); }
.             { cout << yytext[0]; }
\n            { cout << '\n'; }
%%

int main()
{
    yylex();
    return 0;
}
```

Output:

```
abc ABC
```

Practical 4: Find the longest word

Input: Input: `Lex finds the longestword here`

Build:

```
flex -o lex.yy.cpp cd_prac_4.l
g++ -std=c++17 lex.yy.cpp -o practical_4
```

Run:

```
printf 'Lex finds the longestword here\n' | ./practical_4
```

Lex program (`cd_prac_4.l`):

```
%{
#include <iostream>
#include <string>
using namespace std;

string longest_word;
int longest_length = 0;
%}

%option noyywrap

%%
[A-Za-z]+     {
                  if (yyleng > longest_length) {
                      longest_length = yyleng;
                      longest_word = yytext;
                  }
              }
.             { }
\n            { }
%%

int main()
{
    yylex();
```

```
    if (longest_word.empty())
        cout << "No word found.\n";
    else
        cout << "Longest word: " << longest_word << '\n';
    return 0;
}
```

Output:

```
Longest word: longestword
```

Practical 5: Classify programming language tokens

Input: Uses `prac5_program.cpp`.

Build:

```
flex -o lex.yy.cpp cd_prac_5.l
g++ -std=c++17 lex.yy.cpp -o practical_5
```

Run:

```
./practical_5 < prac5_program.cpp
```

Input file (`prac5_program.cpp`):

```
// Sample input for practical 5: Lex token classification
int main() {
    float price = 12.5;
    if (price > 10) {
        return 1;
    }
    return 0;
}
```

Lex program (`cd_prac_5.l`):

```
%{
#include <iostream>
using namespace std;
%}

%option noyywrap
```

```
%x COMMENT

%%
"//"[^\n]* {
    cout << "Comment: " << yytext << '\n';
}

"/*" {
    cout << "Comment: " << yytext;
    BEGIN(COMMENT);
}

<COMMENT>"*/" {
    cout << yytext << '\n';
    BEGIN(INITIAL);
}

<COMMENT>\n {
    cout << '\n';
}

<COMMENT>. {
    cout << yytext[0];
}

(if|else|while|for|do|switch) {
    cout << "Keyword: " << yytext << '\n';
}

(case|break|continue|return) {
    cout << "Keyword: " << yytext << '\n';
}

(int|float|double|char|void|bool|const) {
    cout << "Keyword: " << yytext << '\n';
}

[0-9]+\.[0-9]+([eE][+-]?[0-9]+)? {
    cout << "Float: " << yytext << '\n';
}

[0-9]+ {
    cout << "Integer: " << yytext << '\n';
}

[A-Za-z_][A-Za-z0-9_]* {
    cout << "Identifier: " << yytext << '\n';
}

("=="|"!="|"<="|">="|"++"|"--") {
```

```
    cout << "Operator: " << yytext << '\n';
}

("&&"|"||"|"+="|"-="|"*="|"/="|"%="|"->") {
    cout << "Operator: " << yytext << '\n';
}

[+*/%=<>!&|^~-] {
    cout << "Operator: " << yytext << '\n';
}

[ \t\r\n]+ {
    /* Ignore whitespace between tokens. */
}

. {
    cout << "Symbol: " << yytext << '\n';
}
%%

int main()
{
    yylex();
    return 0;
}
```

Output:

```
Comment: // Sample input for practical 5: Lex token classification
Keyword: int
Identifier: main
Symbol: (
Symbol: )
Symbol: {
Keyword: float
Identifier: price
Operator: =
Float: 12.5
Symbol: ;
Keyword: if
Symbol: (
Identifier: price
Operator: >
Integer: 10
Symbol: )
Symbol: {
Keyword: return
Integer: 1
Symbol: ;
```

```
Symbol: }
Keyword: return
Integer: 0
Symbol: ;
Symbol: }
```

Practical 6: Count words, characters, blank spaces, and lines in a C/C++ source file

Input: Uses `prac6_program.cpp`.

Build:

```
flex -o lex.yy.cpp cd_prac_6.l
g++ -std=c++17 lex.yy.cpp -o practical_6
```

Run:

```
./practical_6 < prac6_program.cpp
```

Input file (`prac6_program.cpp`):

```
#include <iostream>

int main() {
    int number = 6;
    std::cout << "Practical 6 sample: " << number << '\n';
    return 0;
}
```

Lex program (`cd_prac_6.l`):

```
%{
#include <iostream>
using namespace std;

int word_count = 0;
int character_count = 0;
int blank_count = 0;
int line_count = 0;
bool last_character_was_newline = true;
%}

%option noyywrap
```

```
%%
[^ \t\r\n]+   { ++word_count; character_count += yyleng;
last_character_was_newline = false; }
[ \t\r]       { ++character_count; ++blank_count;
last_character_was_newline = false; }
\n            { ++character_count; ++line_count;
last_character_was_newline = true; }
%%

int main()
{
    yylex();
    if (character_count > 0 && !last_character_was_newline)
        ++line_count;

    cout << "Words: " << word_count << '\n';
    cout << "Characters: " << character_count << '\n';
    cout << "Blank spaces: " << blank_count << '\n';
    cout << "Lines: " << line_count << '\n';
    return 0;
}
```

Output:

```
Words: 22
Characters: 129
Blank spaces: 28
Lines: 7
```

Practical 7: Generate a C program that prints successively shorter prefixes

Input: The input string `abcd` is embedded in the Lex specification.

Build:

```
flex -o lex.yy.cpp cd_prac_7.l
g++ -std=c++17 lex.yy.cpp -o practical_7
```

Run:

```
./practical_7
```

Lex program (`cd_prac_7.l`):

```
%{
#include <stdio.h>
#include <string.h>
%}

%%
.|\n    { }
%%

int main()
{
    char str[] = "abcd";
    int i;

    for (i = strlen(str); i > 0; i--)
    {
        printf("%.*s\n", i, str);
    }

    return 0;
}

int yywrap()
{
    return 1;
}
```

Output:

```
abcd
abc
ab
a
```
